



BACKEND DEVELOPER
PORTFOLIO

정관우

신입 백엔드 개발자

한 줄 소개

Spring Boot 기반 백엔드 API를 구현하며 외부 서비스 연동, 상태 관리, 데이터 저장 흐름을 경험한 신입 백엔드 개발자

Contact

Email rhksdn1654@gmail.com
GitHub [GitHub Profile](#)

Education

- 동신대학교 졸업 디지털 콘텐츠 학과
- SSAFY 삼성 청년 SW-AI 아카데미 14기

Certification

- 정보처리기사
- SQLD
- TOEIC® Speaking IM2

PROFILE OVERVIEW

주요 경험

- Spring Boot 기반 REST API 개발
- RabbitMQ 기반 비동기 메시지 처리 흐름 경험
- 트랜잭션을 활용한 데이터 정합성 관리 경험
- Docker Compose, Nginx, Jenkins 기반 배포 환경 경험
- 외부 API 연동 및 응답 데이터 저장 흐름 구현
- Redis를 활용한 데이터 버퍼링 및 임시 저장 경험
- Outbox Pattern, 멍등성 등 실패 재처리 구조 학습

Project History

2026.04 ~ 2026.05 **WEDGE**
AI 기반 웹 UX 진단 서비스

2026.02 ~ 2026.04 **DonDone**
근무 기록 기반 급여 지급 서비스

2026.01 ~ 2026.02 **DNC**
당뇨 환자용 혈당·식단 관리 앱

2025.12 ~ 2025.12 **My Trip Quest**
게임이피케이션 기반 여행 퀘스트 서비스

Portfolio Focus

비동기 처리 / Outbox-Retry / 중복 방어

근무 기록 / 정정 요청 / 승인·반려 흐름

CGM·Dexcom / AI 식단 / Daily Report

관광 API / 위치·사진 인증 / 퀘스트 완료

프로젝트에서 사용한 기술과 역할

백엔드 핵심 활용 기술



Java
Spring Boot 백엔드 프로젝트의 주 언어로 사용



MySQL
DNC의 사용자·혈당·식단·리포트 데이터 저장에 사용



Spring Data JPA
엔티티·Repository 기반 도메인 데이터 처리 경험



Redis
DNC의 CGM 혈당 데이터 임시 저장·버퍼링에 사용

프로젝트 구현 과정에서 주로 사용한 기술



Spring Boot
Controller·Service·Repository/Mapper 계층 기반 기능 구현



PostgreSQL
WEDGE의 실행·분석 상태, DonDone의 근무·승인 데이터 관리에 사용



MyBatis
My Trip Quest에서 SQL 기반 조회·저장 로직 처리



RabbitMQ
WEDGE의 비동기 작업 요청·메시지 전달 흐름에 사용

프로젝트 연동·운영 경험 기술



Docker / Docker Compose
팀 프로젝트의 컨테이너 실행 환경과 Compose 구성 경험



Jenkins
서비스 변경 감지·승인·배포 결과 공유를 포함한 팀 배포 파이프라인 경험



Vue
사용자·관리자 화면과 API 연동 기반 상태 처리 경험



Amazon S3
이미지 업로드 및 파일 저장 흐름 경험

협업·연동·배포 과정에서 사용하거나 구조를 이해한 기술



NGINX
Frontend·Backend·AI Server 간 Reverse Proxy 라우팅 구조 이해



FastAPI
Spring 백엔드와 AI 분석 서버 간 API 연동 경험



React Native
모바일 화면과 백엔드 API 간 데이터 흐름 이해



Firebase Cloud Messaging(FCM)
사용자 설정 기반 모바일 푸시 알림 연동 구조 이해

Wedge

브라우저 실행 기반
UX 진단 리포트 서비스

개발 기간 2026.04 ~ 2026.05 약 7주

플랫폼 Web

개발 인원 6명

담당 범위 Spring Boot 백엔드

- Run / Runner 실행 상태 관리
- Analyzer 입력 구성 및 비동기 요청 처리
- 분석 결과 기반 Report 생성·상태 반영

저장소 GitHub Repository

개발 환경

Backend Java 17, Spring Boot 3.5

Persistence MyBatis

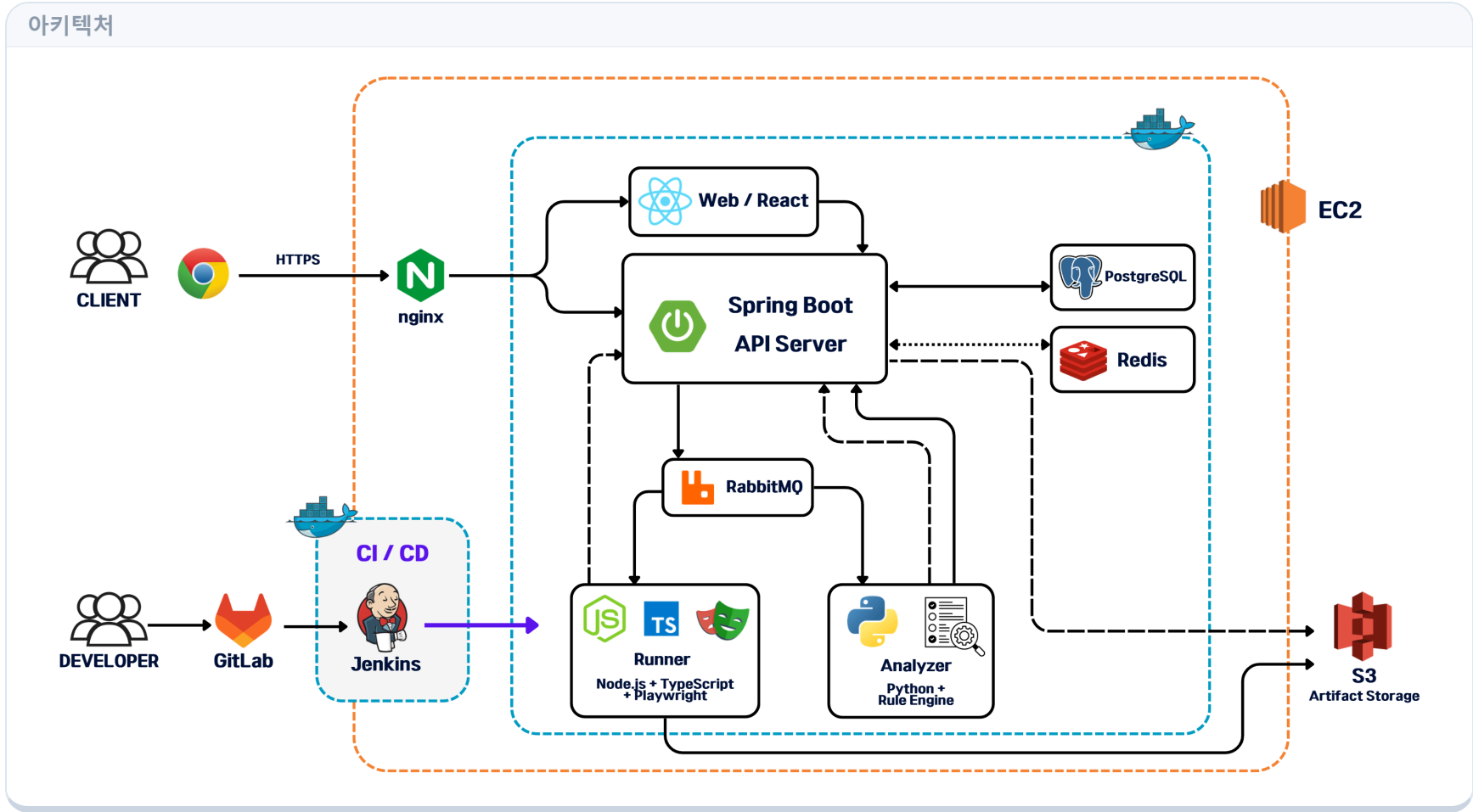
Web React, Vite, TypeScript

Runner Node.js, Playwright

Analyzer FastAPI

DB/MQ PostgreSQL, Redis, RabbitMQ

Infra Docker, Nginx, Jenkins, S3



Run과 Runner 실행 상태 관리

- Run 생성 및 Runner 실행 시작 처리
- Runner 실행 단계에 따른 Run 상태 변경 처리
- Runner callback 수신 처리
- 단계별 이벤트, 체크포인트, 캡처 결과 저장
- Runner 실행 실패 시 실패 상태 저장 및 후속 처리

리포트 생성 및 상태 반영

- 완료된 Analyzer 결과를 기반으로 리포트 생성
- 동일 Run에 대한 리포트 반복 생성 방지
- 최신 리포트 상태 반영
- Analyzer 실패 시 실패 상태 저장 및 사용자-facing 실패 응답 처리

Analyzer 입력 데이터 구성

- Runner 실행 결과를 기반으로 Analyzer 입력 데이터 생성
- Analyzer 요청 시점의 입력 데이터 snapshot 저장
- Analyzer가 필요한 데이터를 조회할 수 있는 내부 API 구성
- 캡처 이미지 접근을 위한 signed URL 응답 처리

비동기 처리 안정성 보강

- Outbox 기반 메시지 발행 상태 저장
- 발행 실패 시 재시도 가능한 작업 상태 관리
- callback 중복 수신 시 상태 반복 반영 방지
- DLQ 메시지 수신 시 Run / AnalysisJob 실패 상태 전환

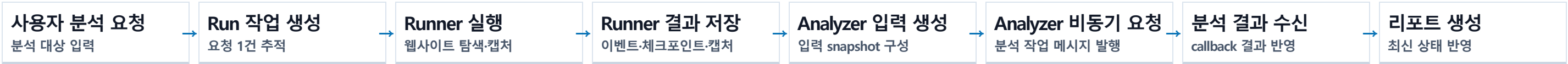
Analyzer 비동기 요청 및 callback 처리

- Analyzer 분석 작업 생성 및 대기 상태 저장
- Analyzer 요청 메시지 생성
- 메시지 큐 발행 실패에 대비한 outbox/retry 처리
- Analyzer의 started/completed/failed callback 처리
- Analyzer callback 중복 수신 방어

Keyword

- Run: 사용자 분석 요청 1건을 추적하는 작업 단위
- Runner: 웹사이트 탐색 및 캡처 실행 프로그램
- Analyzer: Runner 결과 기반 분석 프로그램

서비스 전체 흐름



분석 요청 메시지 유실 위험 완화

문제

- 분석 작업은 DB에 생성됐지만, MQ 발행 실패 시 Analyzer가 작업을 받지 못할 수 있음

원인

- DB 저장과 MQ 발행이 서로 다른 시스템에서 처리됨
- 저장 성공 이후 메시지 발행 실패 시 상태 불일치 발생 가능

해결

- 분석 작업 생성과 outbox 저장을 같은 DB transaction으로 처리
- MQ 발행 실패 메시지는 실패 상태로 기록하고 재시도 대상으로 관리

결과

- DB 저장과 MQ 발행 사이의 실패 지점 추적 가능
- 분석 요청 메시지 유실 위험 완화

Runner / Analyzer callback 중복 수신 방어

문제

- callback 중복 수신 시 Run 상태 변경이나 결과 저장이 중복 처리될 수 있음

원인

- 외부 실행 프로그램 callback은 네트워크 재시도나 worker 재처리로 중복 도착 가능
- 동일 이벤트가 여러 번 처리되면 상태와 결과가 중복 반영될 수 있음

해결

- callback eventId 기준으로 처리 여부 확인
- 이미 처리된 eventId는 상태 변경과 결과 저장을 건너뛸

결과

- 동일 이벤트 재수신에 따른 중복 상태 변경 방어
- 외부 callback 재시도 상황에서도 정상 처리 결과만 반영

Runner 실행 실패 상태 반영

문제

- Runner 실행 요청이 처리되지 못하면 Run이 시작 대기 상태에 머무를 수 있음

원인

- DLQ로 이동한 실행 실패 메시지가 사용자-facing Run 상태와 연결되지 않을 수 있음

해결

- Runner 실행 요청 DLQ listener에서 runId 추출
- 시작 대기 중인 Run만 실패 상태로 변경하고 실패 정보를 저장

결과

- Runner 실행 요청 실패를 Run 상태에 반영
- 사용자가 대기 상태에 머무르지 않고 실패 상태를 확인 가능

안정성 개선 요약

MQ 발행 실패

Outbox / Retry로 유실 위험을 추적하고 재시도

Callback 중복

eventId 기준으로 중복 상태 변경과 결과 저장 방어

Runner 실행 실패

DLQ 메시지를 Run 실패 상태와 failure 정보로 연결

당낭콩(DNC)

CGM·식단 기록 기반
혈당 참고 보조 서비스

개발 기간 2026.01 ~ 2026.02 약 5주

플랫폼 Mobile App

개발 인원 6명

담당 범위 Spring Boot 백엔드

- CGM 데이터 수집·저장 흐름
- AI 식단 분석 결과 매핑·기록 연결
- Daily Report 생성·조회 처리

저장소 GitHub Repository

개발 환경

Backend Java 17, Spring Boot 3.5

Persistence Spring Data JPA

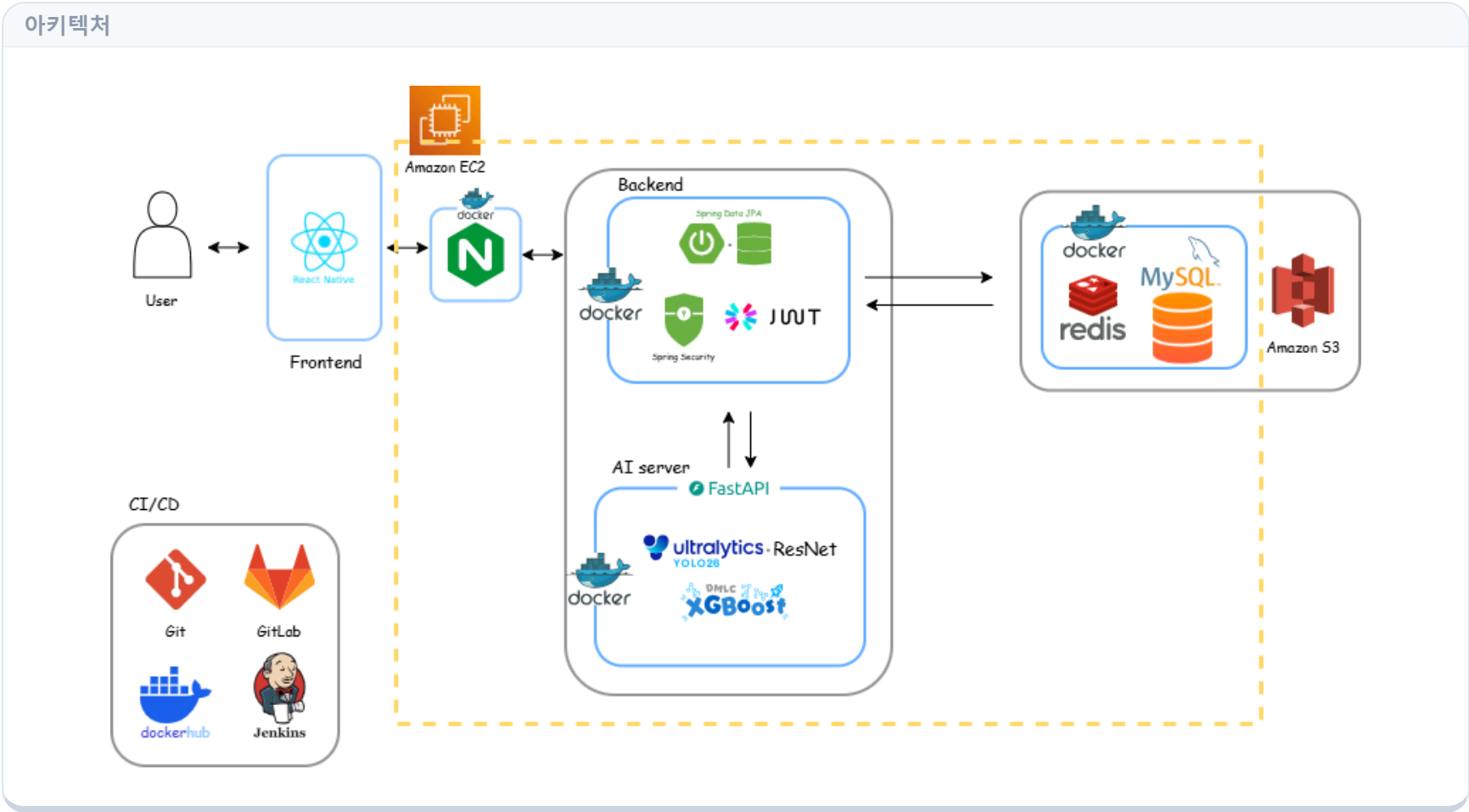
AI Server FastAPI, YOLOv8

Mobile React Native, Expo, TypeScript

DB/Cache MySQL, Redis

Infra Docker, Nginx, Jenkins

External Dexcom OAuth, FCM, S3



AI 식단 분석 및 코칭 연동

- 식단 이미지 분석 요청 처리
- AI 서버의 음식 분석 결과 수신 및 내부 데이터 구조 매핑
- 음식 메타데이터 기반 영양 정보 조회
- 음식 양 기준(Q1~Q5)에 따른 예상 중량 보정
- 분석 결과와 식단 기록 연결
- Gemini를 활용한 식단 코칭 문구 생성 흐름 연동

이미지 저장 및 접근 처리

- 식단 이미지 저장 흐름 보완
- S3 Presigned URL 기반 이미지 접근 처리
- 식단/리포트 조회 시 저장된 이미지 URL 변환 처리

CGM 센서/혈당 데이터 처리 보완

- 사용자별 센서 데이터 격리 로직 보완
- 센서 중복 생성/조회 문제 수정
- 센서 예열 상태(PENDING) 처리 추가
- Dexcom OAuth 연동 직후 혈당 데이터 조회/저장 흐름 보완
- 센서 이력 조회 API 추가
- 센서별 혈당 리포트 필터링 처리

인증/OAuth 및 API 안정화

- 소셜 로그인 실패 케이스 수정
- 이미지 분석 요청의 인증 누락 문제 수정
- 컨트롤러 파라미터 명시로 API 실행 오류 수정
- 센서 응답 DTO 날짜 포맷 수정
- AI 서버 연동 경로 및 응답 데이터 매핑 오류 수정

일일 리포트 생성 및 조회 처리

- 혈당 데이터와 식단 기록 기반 Daily Report 생성 로직 구현
- Daily Report 저장 구조 구성
- 스케줄러 기반 일일 리포트 생성 처리
- Gemini 기반 리포트 요약 문구 생성 연동
- 최신 리포트 및 센서별 리포트 조회 API 구성

Keyword

- CGM: 연속 혈당 측정 데이터
- Dexcom: CGM 혈당 데이터를 제공하는 외부 연동 서비스
- AI 서버: 음식 이미지 분석 및 혈당 변화 참고값 계산 FastAPI 서버
- Gemini: 식단 코칭/리포트 문구 생성을 위한 생성형 AI
- Daily Report: 하루 혈당·식단 요약 리포트
- S3 Presigned URL: 제한 시간 이미지 접근 임시 URL

서비스 전체 흐름



사용자별 혈당 데이터 격리 문제

문제

- Dexcom 샌드박스 환경에서는 여러 사용자가 동일한 테스트 deviceId를 받을 수 있음
- deviceId를 전역 식별자처럼 사용하면 사용자 간 혈당 데이터가 섞일 수 있음

원인

- 샌드박스 deviceId가 사용자별 고유값으로 보장되지 않음
- 센서 조회/저장 로직 일부가 deviceId 기준에 의존

해결

- 센서 조회와 혈당 저장 기준을 현재 사용자 범위로 제한
- 외부 deviceId는 참고값으로 두고 내부 센서 매핑은 사용자 단위로 처리

결과

- 동일 deviceId가 들어와도 사용자별 센서/혈당 데이터 분리
- 리포트 조회 시 사용자 소유 센서 범위 안에서 데이터 조회 가능

AI 서버 연동 계약 불일치

문제

- Spring 백엔드에서 AI 서버로 보낸 음식 분석/혈당 예측 요청이 정상 처리되지 않음
- AI 서버 응답 결과가 Spring DTO에 매핑되지 않음

원인

- Spring 백엔드의 호출 endpoint가 실제 FastAPI 경로와 다름
- AI 서버 응답 JSON 필드명이 Spring DTO 필드와 일치하지 않음

해결

- AI 서버 호출 경로를 실제 FastAPI endpoint에 맞게 수정
- 응답 JSON의 result 값을 Spring DTO의 예측 결과 필드로 매핑

결과

- Spring 백엔드에서 AI 서버 응답 결과를 DTO에 매핑 가능
- AI 서버 endpoint/응답 필드 변경 시 확인할 계약 지점 명확화

Controller 파라미터 바인딩 오류

문제

- 일부 API 호출 시 Controller 파라미터를 인식하지 못해 500 에러 발생

원인

- @PathVariable, @RequestParam에 파라미터 이름이 명시되지 않음
- 컴파일 환경에서 파라미터 이름 추론에 의존한 바인딩이 실패

해결

- 주요 Controller의 @PathVariable, @RequestParam에 명시적 이름 추가
- 여러 도메인 API에서 동일한 파라미터 선언 방식 수정

결과

- 요청 파라미터가 Controller 메서드 인자에 명확하게 매핑
- 파라미터명 추론 실패로 인한 API 500 가능성 감소

트러블슈팅 요약

deviceId 중복
사용자 범위 센서 매핑으로 혈당 데이터 격리

AI API 계약 불일치
FastAPI endpoint와 응답 필드 기준에 맞춰 DTO 매핑

파라미터 바인딩 실패
Controller 파라미터 이름 명시로 API 500 가능성 감소

Other Projects

대표 프로젝트 외 백엔드 구현 경험을 보조 사례로 정리했습니다.

DonDone

2026.02 ~ 2026.04 약 6주 | 5인 프로젝트

GitHub Repository

외국인 근로자의 근무 기록을 급여 확인·
미리받기 흐름과 연결한 급여 지급 서비스

개발 환경

- Backend: Java 17, Spring Boot 3.2
- Persistence: Spring Data JPA
- Web / Mobile: React, Vite, Android Kotlin
- DB/Cache: PostgreSQL, Redis
- Blockchain: Solidity, Foundry, Web3j
- Infra: Docker, Nginx, Jenkins

프로젝트 기여 범위

- Spring Boot 기반 백엔드 API 구현
- 고용주·근로자·관리자 흐름 중 일부 기능 구현

주요 구현

- WorkProof 출퇴근 기록 및 근무 정정 요청 흐름 구현
- 근무 정정 요청에 대한 고용주 승인 / 반려 흐름 구현
- 회사 등록 코드 기반 근로자 등록 흐름 구현
- 관리자·고용주용 승인 상태 조회 API 구현

핵심 처리 흐름

- 출퇴근 기록 → 정정 요청 → 고용주 검토 → 승인 / 반려 → 근무 기록 확정 → 급여·미리받기 흐름 연계

My Trip Quest

2025.12 ~ 2025.12 약 4주 | 2인 프로젝트

GitHub Repository

관광지 방문을 위치·사진 인증 기반 퀘스트와
보상 구조로 연결한 여행 게이미피케이션
서비스

개발 환경

- Backend: Java 17, Spring Boot 3.5
- Persistence: MyBatis
- Web: Vue 3, Vite
- DB: MySQL
- Auth: JWT, OAuth2
- External: 한국관광공사 API

프로젝트 기여 범위

- Spring Boot 백엔드 API와 Vue 화면 구현
- 외부 관광 API 연동 및 퀘스트 완료 흐름 구현

주요 구현

- Spring Boot / MyBatis 기반 백엔드 API 구현
- 한국관광공사 API 연동 및 관광지 데이터 조회·저장 흐름 구현
- 위치 / 사진 인증 기반 퀘스트 완료 흐름 구현
- 포인트·랭킹 등 게이미피케이션 기능 일부 구현

핵심 처리 흐름

- 관광지 데이터 연동 → 퀘스트 구성 → 위치·사진 인증 → 퀘스트 완료 → 포인트·랭킹 반영